

LAPORAN AKHIR MAGANG MANDIRI

Rancang Bangun Sistem Portal Berita dan Content Management System (CMS) Redaksi Di PT. Ketik Media Siber

No. PKS: 308/UN63.7/IA-IF/2025

Diajukan untuk memenuhi persyaratan kelulusan
Program Magang Mandiri

oleh :

Farrel Aqeel Danendra / 23081010204

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

**UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA
TIMUR**

2026

BAB III

RANCANG BANGUN SISTEM PORTAL BERITA DAN CMS

REDAKSI

III.1 Deskripsi Persoalan

PT. Ketik Media Siber merupakan perusahaan yang bergerak di bidang media digital dan mengelola portal berita Klojen.com. Sebelum adanya sistem baru, proses pengelolaan konten berita dilakukan secara manual dengan alur kerja yang kurang terstruktur. Para jurnalis menulis berita, kemudian menyerahkan draf kepada editor melalui komunikasi informal, dan editor melakukan proses publikasi secara langsung tanpa sistem pelacakan revisi yang memadai. Kondisi ini menyebabkan beberapa permasalahan, antara lain: kesulitan dalam melacak riwayat perubahan artikel, tidak adanya mekanisme penjadwalan publikasi artikel, ketiadaan kontrol akses berbasis peran (role-based access control), serta tidak adanya sistem pencarian yang efisien untuk menemukan artikel yang telah dipublikasikan.

Berdasarkan permasalahan tersebut, penulis ditugaskan untuk melakukan perancang bangun Sistem Portal Berita dan Content Management System (CMS) Redaksi yang mampu mengatasi seluruh kendala di atas. Sistem yang dikembangkan harus memenuhi kebutuhan berikut:

1. **Perancangan Arsitektur Sistem** — menyusun struktur proyek secara keseluruhan, meliputi pemilihan tech stack, konfigurasi repository, perancangan database (CDM, PDM, ERD), serta dokumentasi desain sistem menggunakan diagram UML.
2. **Fitur Cronjob Scheduled Publish** — membangun mekanisme penjadwalan publikasi artikel secara otomatis menggunakan cron job, di mana artikel yang telah diatur waktu tayangnya akan otomatis dipublikasikan ketika jadwal tiba.
3. **Fitur Manajemen User (CRUD)** — membangun fitur pengelolaan pengguna sistem oleh admin, meliputi pembuatan akun baru, pengeditan data, aktivasi/nonaktifkan akun, serta penghapusan pengguna dengan penanganan dampak terhadap data terkait.

III.2 Proses Pelaksanaan

III.2.1 Penyusunan Struktur Proyek dan Perancangan Database

Tahap pertama dalam pelaksanaan proyek adalah menyusun struktur proyek dari awal (from scratch) serta melakukan perancangan database yang komprehensif. Penulis bertanggung jawab dalam menentukan arsitektur sistem, memilih teknologi yang digunakan, serta merancang seluruh struktur database yang menjadi fondasi sistem.

Arsitektur sistem dirancang menggunakan pendekatan **decoupled architecture**, di mana backend dan frontend dipisah menjadi dua aplikasi terpisah yang berkomunikasi melalui REST API. Backend dibangun menggunakan **Laravel 12** dengan bahasa pemrograman PHP 8.4 dan database SQLite. Frontend dibangun menggunakan **Next.js 15** dengan TypeScript dan framework CSS Tailwind CSS. Pendekatan ini memungkinkan pengembangan kedua sisi berjalan secara paralel dan memudahkan maintenance di kemudian hari.

Pada sisi backend, penulis menerapkan pola **Service-Repository Pattern** yang memisahkan logika bisnis (Service) dari operasi database (Repository). Setiap entitas utama memiliki Service dan Repository tersendiri, seperti `ArticleService`, `UserService`, `CategoryService`, dan lain-lain. Seluruh operasi database dilakukan melalui Query Builder Laravel, bukan ORM Eloquent, untuk memberikan kontrol penuh terhadap query SQL yang dieksekusi.

Perancangan database dimulai dari **Conceptual Data Model (CDM)**, kemudian diturunkan menjadi **Physical Data Model (PDM)**, dan selanjutnya diimplementasikan dalam bentuk file migrasi Laravel. Database sistem terdiri dari 15 tabel utama:

No	Nama Tabel	Deskripsi
1	users	Data pengguna sistem (admin, editor, journalist, reader)
2	categories	Kategori berita (Politik, Ekonomi, Olahraga, dsb.)
3	tags	Tag artikel untuk pengelompokan topik
4	articles	Artikel berita dengan metadata lengkap

5	article_tags	Relasi many-to-many antara artikel dan tag
6	article_revisions	Riwayat revisi artikel (snapshot before update)
7	scheduled_articles	Jadwal publikasi artikel terjadwal
8	search_indexes	Index pencarian full-text
9	comments	Komentar pembaca pada artikel
10	media	File media (gambar) yang diunggah
11	bookmarks	Artikel yang disimpan oleh pembaca
12	refresh_tokens	Token refresh untuk autentikasi JWT
13	password_reset_tokens	Token reset password
14	page_views	Pencatatan statistik kunjungan halaman
15	comment_rate_limits	Pembatasan rate komentar per IP

Seluruh tabel menggunakan UUID sebagai primary key bertipe string untuk keamanan dan menghindari prediksi ID. Relasi antar tabel dirancang dengan foreign key constraint yang menjaga integritas data.

III.2.2 Implementasi Cronjob Scheduled Publish

Fitur *Scheduled Publish* memungkinkan editor atau admin untuk mengatur waktu publikasi artikel di masa depan. Ketika waktu yang dijadwalkan tiba, sistem secara otomatis mengubah status artikel dari `scheduled` menjadi `published` tanpa intervensi manual.

Implementasi fitur ini melibatkan tiga komponen utama: (1) **Artisan Command** sebagai entry point yang dipanggil oleh scheduler, (2) **ScheduledPublishService** yang berisi seluruh logika bisnis penjadwalan, dan (3) **Laravel Task Scheduler** yang dikonfigurasi untuk menjalankan command setiap menit.

Berikut adalah potongan kode dari Artisan Command yang menjadi trigger utama proses scheduled publish:

```
PHP
```

```

class PublishScheduledArticles extends Command
{
    protected $signature = 'articles:publish-scheduled';
    protected $description = 'Publish semua artikel terjadwal yang sudah waktunya tayang';

    public function __construct(private readonly ScheduledPublishService $service)
    {
        parent::__construct();
    }

    public function handle(): int
    {
        $this->info('[ScheduledPublish] Memulai proses auto-publish...');
        $count = $this->service->run();

        if ($count === 0) {
            $this->line(' → Tidak ada artikel yang perlu dipublish.');
```

Potongan kode 1. Artisan Command `articles:publish-scheduled` yang menjadi entry point cron job.

Pada potongan kode di atas, class `PublishScheduledArticles` merupakan Artisan Command dengan signature `articles:publish-scheduled`. Command ini menerima dependency `ScheduledPublishService` melalui constructor injection. Ketika dijalankan, method `handle()` memanggil `$this->service->run()` yang mengembalikan jumlah artikel yang berhasil dipublish. Command ini kemudian menampilkan output ke console berupa jumlah artikel yang berhasil diproses.

Inti dari proses penjadwalan terdapat pada method `run()` di `ScheduledPublishService`. Berikut adalah potongan kode yang melakukan query untuk mengambil artikel yang sudah waktunya tayang:

PHP

```

// Query artikel terjadwal yang sudah waktunya tayang
$due = DB::table('scheduled_articles')
```

```

->where('scheduled_at', '<=', now())
->where('is_published', false)
->get();

if ($due->isEmpty()) {
    return 0; // Tidak ada artikel yang perlu dipublish
}

```

Potongan kode 2. Query pengambilan artikel terjadwal yang sudah jatuh tempo.

Kode di atas melakukan query ke tabel `scheduled_articles` dengan kondisi `scheduled_at <= NOW()` (waktu jadwal sudah lewat atau sama dengan waktu sekarang) dan `is_published = false` (belum diproses sebelumnya). Jika tidak ada artikel yang memenuhi kondisi, method langsung mengembalikan nilai 0.

Setelah mendapatkan daftar artikel yang perlu dipublish, setiap artikel diproses dalam sebuah database transaction untuk menjamin konsistensi data. Berikut adalah potongan kode proses publish:

PHP

```

// Proses publish satu artikel dalam database transaction
DB::transaction(function () use ($scheduled, $articleId, $now) {

    // Update status artikel menjadi 'published'
    DB::table('articles')
        ->where('id', $articleId)
        ->where('status', 'scheduled') // double-check status
        ->update([
            'status'          => 'published',
            'published_at'    => $now,
            'updated_at'      => $now,
        ]);

    // Tandai jadwal sudah diproses
    DB::table('scheduled_articles')
        ->where('id', $scheduled->id)
        ->update(['is_published' => true]);

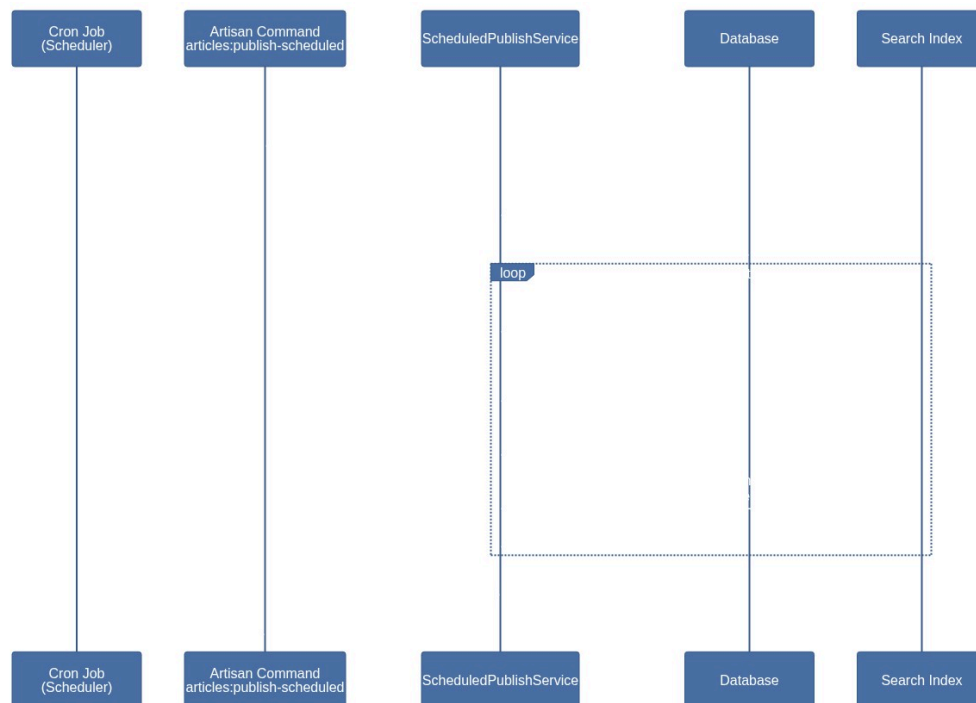
    // Reindex ke tabel search_indexes
    $this->upsertSearchIndex($articleId, $now);
});

```

Potongan kode 3. Proses publish artikel dalam database transaction.

Potongan kode di atas menunjukkan bahwa setiap artikel diproses dalam satu transaction atomik yang terdiri dari tiga operasi: (1) update status artikel menjadi `published` beserta timestamp `published_at`, (2) update flag `is_published` pada tabel `scheduled_articles` agar tidak diproses ulang, dan (3) reindex artikel ke tabel `search_indexes` agar langsung dapat ditemukan melalui fitur pencarian. Penggunaan `DB::transaction()` memastikan bahwa jika salah satu operasi gagal, seluruh perubahan akan di-rollback sehingga tidak terjadi inkonsistensi data.

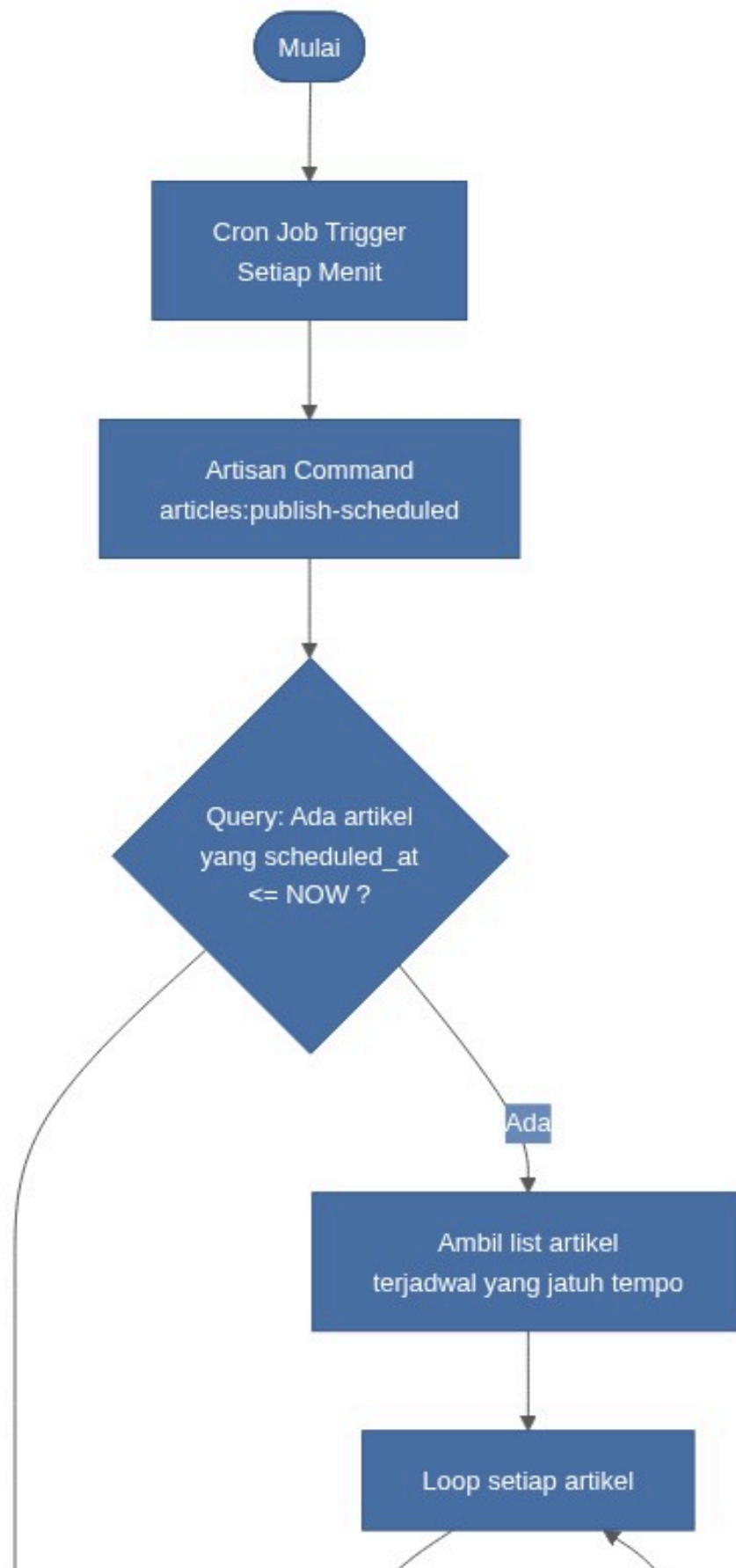
Berikut adalah Sequence Diagram yang menggambarkan alur kerja fitur Scheduled Publish secara keseluruhan:

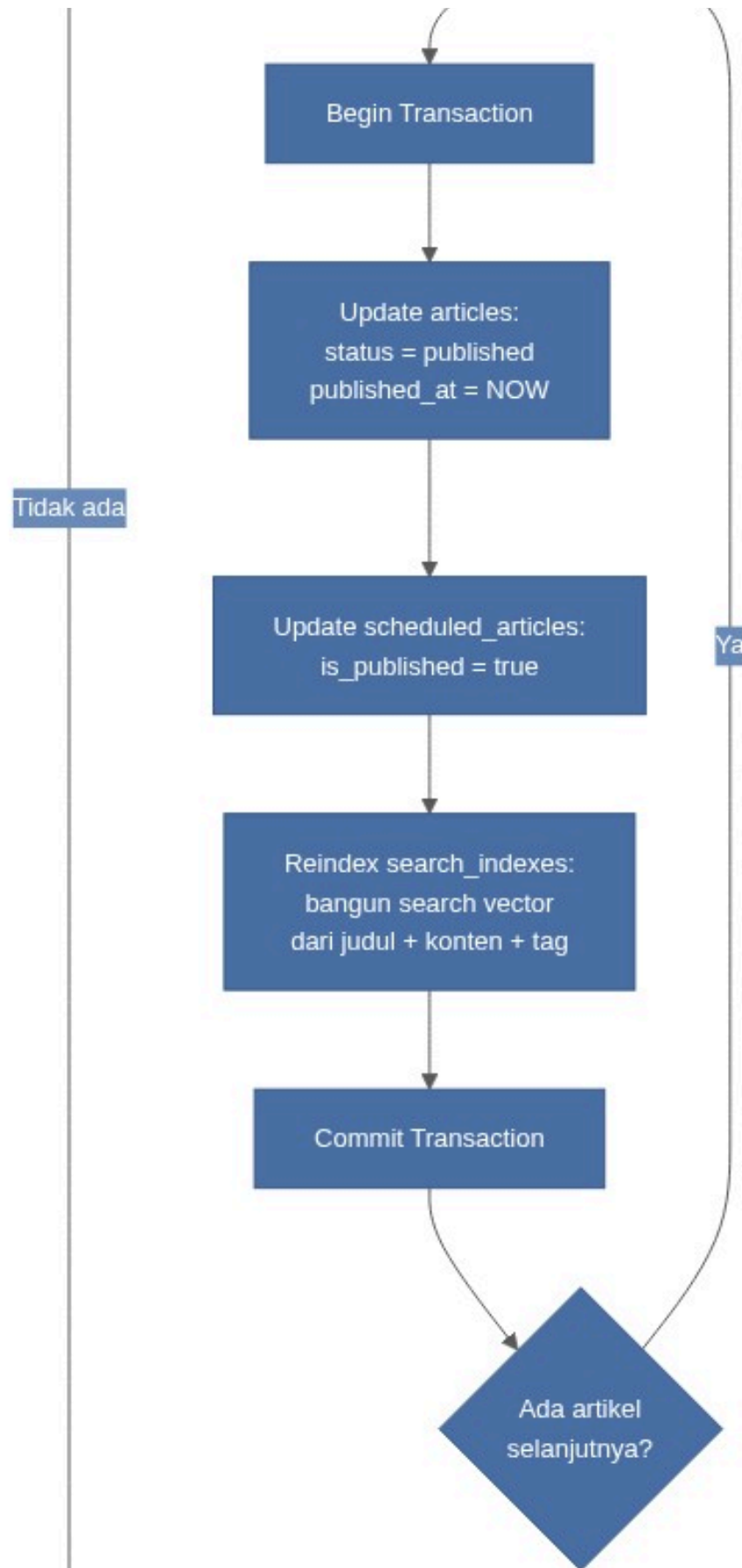


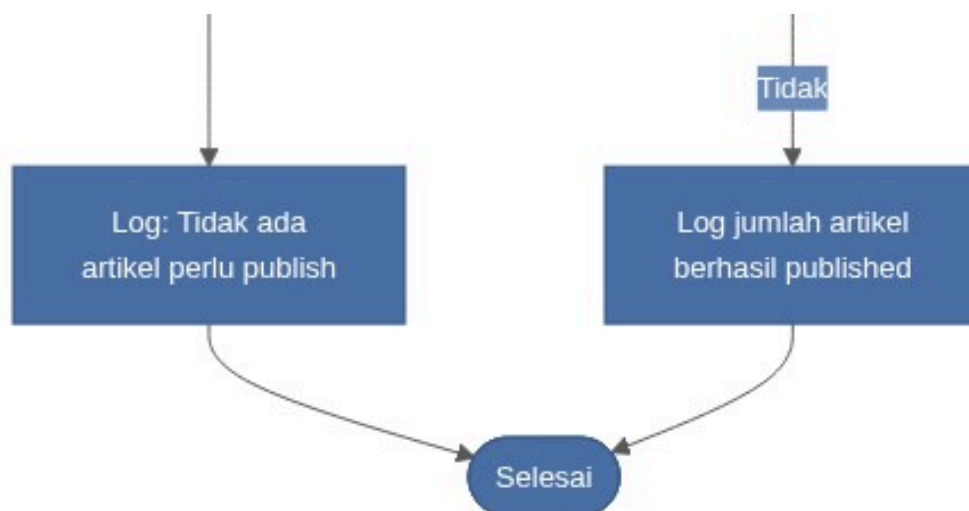
Gambar 3.1 Sequence Diagram — Cronjob Scheduled Publish

Diagram di atas menunjukkan alur lengkap dimulai dari Cron Job yang men-trigger Artisan Command setiap menit, kemudian command memanggil ScheduledPublishService untuk melakukan query artikel yang jatuh tempo, lalu memproses setiap artikel dalam transaction yang mencakup update status artikel, update flag jadwal, dan reindex search index.

Berikut adalah Activity Diagram yang menggambarkan alur aktivitas fitur Scheduled Publish:







Gambar 3.2 Activity Diagram — Scheduled Publish

Activity Diagram di atas menggambarkan proses secara detail dimulai dari trigger cron job, pengecekan artikel yang jatuh tempo, loop pemrosesan setiap artikel, hingga logging hasil eksekusi. Setiap artikel diproses dalam satu transaction yang menjamin atomisitas operasi.

III.2.3 Implementasi Manajemen User (CRUD) pada Role Admin

Fitur Manajemen User merupakan fitur yang hanya dapat diakses oleh pengguna dengan role **Admin**. Fitur ini mencakup operasi Create (membuat user baru), Read (melihat daftar dan detail user), Update (mengedit data user), dan Delete (menghapus user). Sistem manajemen user dirancang dengan memperhatikan aspek keamanan, seperti pencegahan admin menghapus akun sendiri, auto-generate password yang aman, dan pengiriman kredensial melalui email.

Arsitektur fitur ini mengikuti pola Service-Repository yang konsisten: **UserController** menangani HTTP request/response, **UserService** menangani logika bisnis, dan **UserRepository** menangani operasi database.

Berikut adalah potongan kode untuk operasi Create User, di mana admin dapat membuat akun baru untuk jurnalis, editor, atau admin lainnya:

PHP

```
// UserService::createUser() - Membuat user baru oleh Admin
public function createUser(array $data): User
{

```

```

// 1. Cek duplikat email
if ($this->userRepository->findByEmail($data['email'])) {
    throw new \RuntimeException('EMAIL_ALREADY_EXISTS', 409);
}

// 2. Generate password sementara: 12 karakter acak
$plainPassword = $this->generatePassword();

// 3. Buat user - password otomatis di-hash via model cast 'hashed'
$user = $this->userRepository->create([
    'name'      => $data['name'],
    'email'     => $data['email'],
    'password'  => $plainPassword,
    'role'     => $data['role'],
    'is_active' => true,
]);

// 4. Kirim email berisi kredensial ke user baru
Mail::to($user->email)->send(new NewUserCredentials(
    name: $user->name, email: $user->email,
    plainPassword: $plainPassword, role: $user->role,
));

return $user;
}

```

Potongan kode 4. Method *createUser()* pada *UserService* untuk membuat user baru.

Method **createUser()** melakukan beberapa langkah penting: pertama, pengecekan duplikat email melalui repository untuk mencegah duplikasi akun. Kedua, generate password acak 12 karakter yang memenuhi kriteria keamanan (huruf besar, huruf kecil, angka, dan simbol). Ketiga, insert data user ke database dimana password secara otomatis di-hash oleh Laravel melalui cast **hashed** pada model. Keempat, pengiriman email berisi kredensial login (email dan password sementara) ke alamat email user baru, sehingga user dapat langsung login dan mengganti password.

Berikut adalah potongan kode untuk generate password acak:

PHP

```

// Generate password acak yang aman (12 karakter)
private function generatePassword(): string
{
    $upper    = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
    $lower    = 'abcdefghijklmnopqrstuvwxyz';
    $digits   = '23456789';

```

```

$symbols = '@#$$%&!';

// Minimal 1 dari setiap jenis karakter
$password = $upper[random_int(0, strlen($upper) - 1)];
$password .= $lower[random_int(0, strlen($lower) - 1)];
$password .= $digits[random_int(0, strlen($digits) - 1)];
$password .= $symbols[random_int(0, strlen($symbols) - 1)];

// Isi sisa 8 karakter dari gabungan semua
$all = $upper . $lower . $digits . $symbols;
for ($i = 0; $i < 8; $i++) {
    $password .= $all[random_int(0, strlen($all) - 1)];
}

return str_shuffle($password); // Acak urutan
}

```

Potongan kode 5. Method `generatePassword()` untuk menghasilkan password acak yang aman.

Fungsi `generatePassword()` menghasilkan password sepanjang 12 karakter dengan menjamin minimal terdapat satu huruf besar, satu huruf kecil, satu angka, dan satu simbol. Karakter-karakter yang membingungkan seperti huruf O, l, 1, dan 0 sengaja dihilangkan dari set karakter untuk menghindari kebingungan saat user menginput password. Delapan karakter sisanya diambil secara acak dari gabungan seluruh set karakter, kemudian keseluruhan karakter diacak urutannya menggunakan fungsi `str_shuffle()`.

Operasi Update User memiliki logika khusus yang mencegah admin mengubah akun sendiri serta melakukan revoke token jika role user diubah:

PHP

```

// UserService::updateUser() - Mengupdate data user oleh Admin
public function updateUser(string $id, array $data, string $currentUserId):
User
{
    // Validasi: admin tidak bisa mengubah akun sendiri
    if ($id === $currentUserId) {
        throw new \RuntimeException('CANNOT_MODIFY_SELF', 403);
    }

    $user = $this->userRepository->findById($id);
    if (!$user) throw new \RuntimeException('USER_NOT_FOUND', 404);

    // Cek duplikat email jika email diubah
    if (isset($data['email']) && $this->userRepository-

```

```

>findByEmailExcept($data['email'], $id)) {
    throw new \RuntimeException('EMAIL_ALREADY_EXISTS', 409);
}

$oldRole = $user->role;
$updateData = array_filter($data, fn($value) => !is_null($value));
$this->userRepository->update($user, $updateData);

// Jika role diubah, revoke semua token (logout paksa)
if (isset($updateData['role']) && $updateData['role'] !== $oldRole) {
    $this->refreshTokenRepository->revokeAllForUser($id);
}

return $user->refresh();
}

```

Potongan kode 6. Method updateUser() dengan validasi dan revoke token otomatis.

Potongan kode di atas menunjukkan beberapa mekanisme penting: (1) validasi bahwa admin tidak dapat mengubah akunnya sendiri (CANNOT_MODIFY_SELF) untuk mencegah admin mengunci dirinya sendiri dari sistem, (2) pengecekan duplikat email dengan pengecualian ID user saat ini (findByEmailExcept), (3) update hanya field yang dikirim oleh client menggunakan array_filter, dan (4) jika role user diubah, seluruh refresh token milik user tersebut di-revoke yang secara efektif memaksa logout dari seluruh perangkat aktif. Mekanisme revoke ini penting untuk memastikan perubahan hak akses langsung berlaku tanpa menunggu token kedaluwarsa.

Operasi Delete User merupakan operasi yang paling kompleks karena melibatkan penanganan dampak terhadap data terkait:

PHP

```

// UserService::deleteUser() - Menghapus user oleh Admin
public function deleteUser(string $id, string $currentUserId): void
{
    // Validasi: tidak bisa menghapus diri sendiri
    if ($id === $currentUserId) {
        throw new \RuntimeException('CANNOT_MODIFY_SELF', 403);
    }

    $user = $this->userRepository->findById($id);
    if (!$user) throw new \RuntimeException('USER_NOT_FOUND', 404);

    // 1. Revoke semua refresh token (logout paksa)
    $this->refreshTokenRepository->revokeAllForUser($id);
}

```

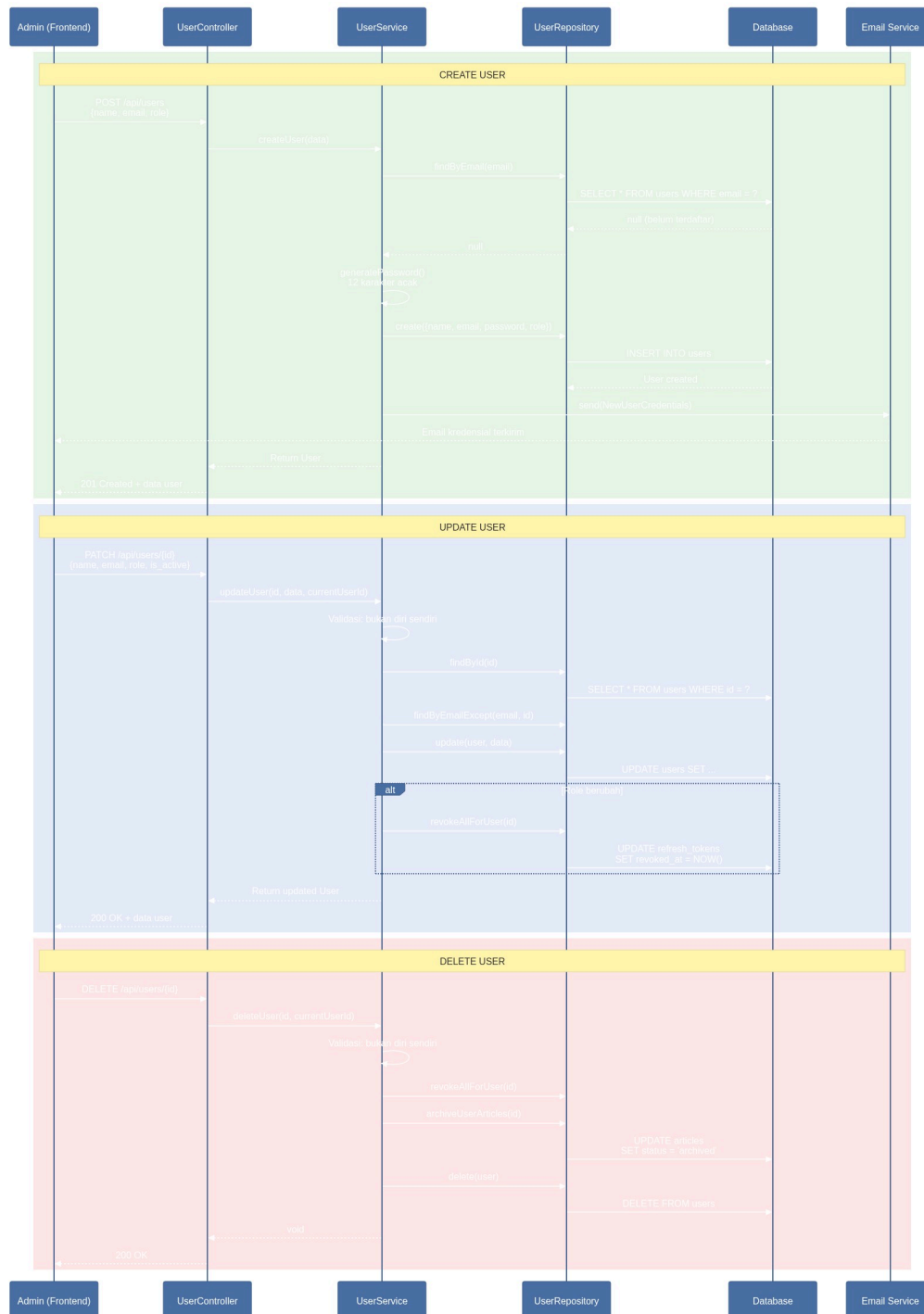
```
// 2. Arsipkan semua artikel milik user
$this->articleRepository->archiveUserArticles($id);

// 3. Hapus user dari database
$this->userRepository->delete($user);
}
```

Potongan kode 7. Method `deleteUser()` dengan penanganan dampak terhadap data terkait.

Method `deleteUser()` melakukan tiga operasi secara berurutan: (1) revoke seluruh refresh token user untuk memaksa logout dari seluruh perangkat, (2) arsipkan seluruh artikel milik user dengan mengubah statusnya menjadi `archived` — hal ini dilakukan alih-alih menghapus artikel untuk menjaga sejarah konten portal, dan (3) hapus data user dari database. Seluruh operasi ini dilakukan setelah validasi bahwa user yang dihapus bukan akun admin sendiri.

Berikut adalah Sequence Diagram yang menggambarkan alur kerja CRUD User secara lengkap:

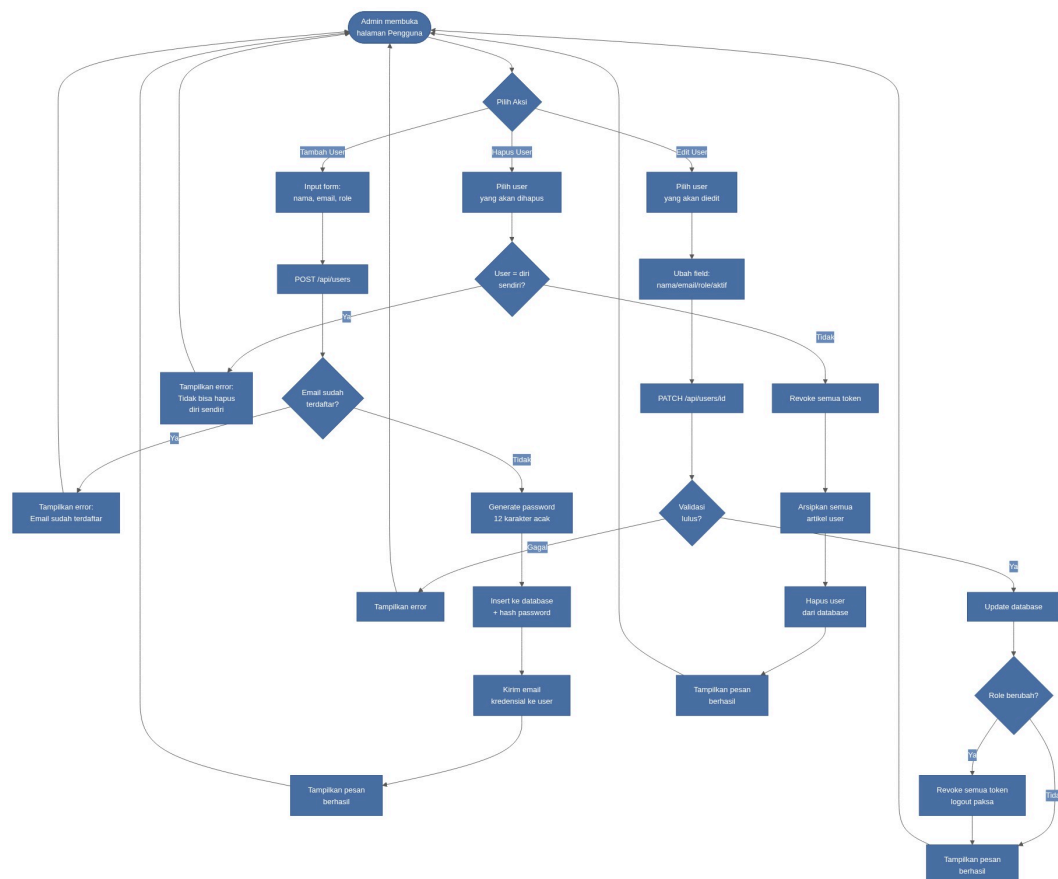


Gambar 3.3 Sequence Diagram — Manajemen User (CRUD)

Diagram di atas menggambarkan tiga skenario operasi CRUD: Create User (ditandai blok hijau) di mana admin menginput data user baru, sistem melakukan pengecekan email, generate password, insert ke database, dan mengirim email kredensial. Update User (blok biru) di mana sistem melakukan validasi, update

data, dan revoke token jika role berubah. Delete User (blok merah) di mana sistem melakukan revoke token, arsipkan artikel, dan hapus user.

Berikut adalah Activity Diagram yang menggambarkan alur aktivitas fitur Manajemen User:



Gambar 3.4 Activity Diagram — Manajemen User

Activity Diagram di atas menggambarkan keseluruhan alur operasi manajemen user dari perspektif admin. Setiap operasi memiliki jalur validasi dan penanganan error tersendiri, seperti pengecekan duplikat email pada operasi Create, validasi bukan diri sendiri pada operasi Update dan Delete, serta mekanisme revoke token pada operasi Update (jika role berubah) dan Delete.

III.3 Pencapaian Hasil

Berdasarkan pelaksanaan proyek yang telah dilakukan, berikut adalah pencapaian yang telah diraih:

1. **Struktur Proyek Berhasil Disusun** — Penulis berhasil menyusun seluruh struktur proyek dari awal, meliputi konfigurasi repository Git, pemilihan dan setup tech stack (Laravel 12, Next.js 15, Tailwind CSS), perancangan database komprehensif (CDM, PDM, ERD) yang terdiri dari 15 tabel, serta dokumentasi perancangan sistem menggunakan diagram UML (Use Case Diagram, Sequence Diagram, Activity Diagram, dan Class Diagram). Seluruh dokumentasi tersimpan rapi dalam repository proyek.
2. **Fitur Cronjob Scheduled Publish Berfungsi** — Fitur penjadwalan publikasi artikel berhasil diimplementasikan dan berjalan dengan baik. Cron job berhasil memproses artikel terjadwal secara otomatis setiap menit, mengubah status artikel menjadi `published`, dan melakukan `reindex search index`. Fitur ini telah diuji dengan skenario penjadwalan artikel minimal 5 menit di masa depan (sesuai business rule BR-04) dan berhasil mempublikasikan artikel tepat waktu.
3. **Fitur Manajemen User (CRUD) Berfungsi** — Seluruh operasi CRUD untuk manajemen user oleh admin berhasil diimplementasikan: pembuatan user baru dengan auto-generate password dan pengiriman email kredensial, pengeditan data user dengan mekanisme revoke token otomatis saat perubahan role, aktivasi/nonaktifkan akun user, serta penghapusan user dengan penanganan dampak (revoke token, arsipkan artikel). Seluruh operasi telah dilengkapi dengan validasi keamanan yang memadai.
4. **Integrasi dengan Sistem Keseluruhan** — Kedua fitur yang dikembangkan telah terintegrasi dengan baik dalam ekosistem sistem CMS. Fitur Scheduled Publish terintegrasi dengan editorial workflow (jurnalisi menulis → editor me-review → jadwalkan → publish otomatis), dan fitur Manajemen User terintegrasi dengan sistem autentikasi JWT serta kontrol akses berbasis peran.

Dalam proses pelaksanaan, penulis juga memperoleh beberapa pengalaman berharga, antara lain:

- Pemahaman mendalam mengenai pola arsitektur Service-Repository dan penerapannya dalam membangun aplikasi yang maintainable dan

testable.

- Pengalaman dalam merancang database yang efisien dengan menggunakan UUID sebagai primary key dan memahami trade-off-nya dibandingkan auto-increment ID.
- Pemahaman mengenai mekanisme cron job dan pentingnya database transaction dalam menjaga konsistensi data.
- Pengalaman dalam menangani edge case pada operasi CRUD, seperti pencegahan modifikasi akun sendiri dan penanganan dampak penghapusan user terhadap data terkait.
- Kemampuan dalam menggunakan Git dan GitHub untuk kolaborasi tim dalam pengembangan proyek.

Sistem yang dibangun telah diuji secara internal oleh tim pengembang dan mentor di PT. Ketik Media Siber. Hasil pengujian menunjukkan bahwa seluruh fitur berjalan sesuai spesifikasi yang telah ditetapkan. Beberapa catatan perbaikan yang diidentifikasi untuk pengembangan selanjutnya antara lain: penambahan fitur audit log untuk melacak seluruh perubahan yang dilakukan admin, implementasi soft delete pada tabel users untuk mempertahankan riwayat, serta penambahan fitur bulk action pada manajemen user untuk efisiensi pengelolaan pengguna dalam jumlah besar.